

Manual de Implantacao do SECP em Producao

Guia definitivo para instalacao, configuracao, validacao e operacao inicial em unidades interessadas

Sistema	SECP - Sistema Eletronico de Controle de Ponto
Ambiente alvo	Servidor Linux Ubuntu com Docker Compose
Data de emissao	16/07/2026
Classificacao	Uso interno - nao incluir senhas reais neste documento

Este manual consolida o procedimento recomendado para implantar o SECP em producao. Ele foi escrito para equipes de infraestrutura e suporte tecnico, com foco em comandos reproduziveis, validacoes objetivas e operacao segura.

1. Escopo e Premissas

O procedimento cobre a implantacao do SECP usando os arquivos Docker presentes no projeto: **compose.prod.yaml**, **Dockerfile**, **.env.production**, PgBouncer, PostgreSQL, Redis, workers e observabilidade opcional.

- A unidade deve possuir servidor Ubuntu dedicado ou VM equivalente, com acesso administrativo.
- O banco principal fica no container **secp-postgres** e a aplicacao usa preferencialmente PgBouncer via **secp-pgbouncer**.
- Workers rodam em containers separados para evitar que rotinas pesadas concorram com a interface web.
- Credenciais, tokens, segredos e URLs internas devem ser definidos localmente pela unidade. Nunca versionar esses valores.

Ponto de atencao: este manual nao substitui politicas institucionais de seguranca, backup, firewall e mudanca. Use janela de manutencao para primeira implantacao, migrations e alteracoes de infraestrutura.

Arquitetura resumida

Componente	Container/servico	Funcao
Aplicacao web	secp-web	Next.js em producao, porta interna 3000.
Banco	secp-postgres	PostgreSQL 18, volume persistente secp_postgres_data.
Pool de conexoes	secp-pgbouncer	PgBouncer na porta 6432, exposto apenas em 127.0.0.1.
Filas/cache	secp-redis	Redis 7 com appendonly habilitado.
Workers	secp-worker-*	AFD, SARH, login SARH, reprocessamento, calendario, relatorios e relgios.
Admin opcional	secp-pgadmin	pgAdmin no profile admin, nao recomendado expor sem controle.

2. Requisitos Minimos

Item	Mínimo recomendado	Observacao
Sistema operacional	Ubuntu Server LTS 22.04 ou 24.04	Manter atualizacoes de seguranca aplicadas.
CPU	4 vCPU	Aumentar se houver muitos relgios, SARH frequente ou relatorios intensos.
Memoria	8 GB RAM	O compose limita web e workers, mas o PostgreSQL tambem precisa folga.
Disco	100 GB SSD	Separar volumes/backups quando possivel. Monitorar crescimento de banco e relatorios.
Rede	Acesso aos relgios, AD, SARH e DNS institucional	Liberar portas conforme topologia da unidade.
Runtime	Docker Engine e Docker Compose v2	Node nao precisa estar instalado no host para producao Docker.

Portas usuais

- **3000/tcp**: SECP web no host, normalmente atras de proxy reverso, IIS/ARR ou DNS interno.
- **3001/tcp**: worker Henry online, quando usado.
- **6432/tcp**: PgBouncer restrito a 127.0.0.1 no compose.
- **5050/tcp**: pgAdmin somente se profile admin for ativado.
- **3100/9090**: Grafana/Prometheus se observabilidade for habilitada e publicada.

3. Preparacao do Servidor Ubuntu

Atualizar pacotes e instalar dependencias

```
sudo apt-get update
sudo apt-get upgrade -y
sudo apt-get install -y ca-certificates curl git openssl unzip sshpass
```

Instalar Docker Engine e Compose v2

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
newgrp docker
docker version
docker compose version
```

Em ambientes com politica restritiva, instale Docker pelo repositório homologado da unidade. O requisito técnico e que **docker compose version** responda com Compose v2.

Criar diretório padrão

```
sudo mkdir -p /opt/secp
sudo chown -R $USER:$USER /opt/secp
cd /opt/secp
```

4. Obter o Código e Preparar Arquivos

Clonar ou atualizar o repositório

```
cd /opt/secp
git clone <URL_DO_REPOSITARIO_SECP> secp-app
cd /opt/secp/secp-app
git checkout <BRANCH_OU_TAG_DE_PRODUCAO>
```

Recomendacao: implantar por tag ou commit fixo. Evite apontar producao diretamente para uma branch movel sem registro de mudanca.

Estrutura esperada

```
/opt/secp/secp-app/
  compose.prod.yaml
  Dockerfile
  .env.production
  docker/pgbouncer/pgbouncer.ini
  docker/pgbouncer/userlist.txt
  docker/oracle/instantclient-basiclite-linuxx64.zip # opcional
  backups/
  observability/
```

5. Configurar .env.production

Crie o arquivo **.env.production** no diretório raiz do projeto. Use valores reais apenas no servidor. O exemplo abaixo usa placeholders.

```
APP_VERSION=20260716_unidade

POSTGRES_USER=secp
POSTGRES_PASSWORD=<SENHA_FORTE_DO_POSTGRES>
POSTGRES_DB=secp_prod
DATABASE_URL=postgresql://secp:<SENHA_FORTE_DO_POSTGRES>@postgres:5432/secp_prod?schema=public
DATABASE_URL_POOLED=postgresql://secp:<SENHA_FORTE_DO_POSTGRES>@pgbouncer:6432/secp_prod?schema=public

AUTH_SECRET=<GERAR_COM_OPENSSL RAND_HEX_32>
NEXTAUTH_SECRET=<MESMO_VALOR_OU_OUTRO_SEGREDO_FORTE>
AUTH_URL=https://<DNS_DO_SECP>
NEXTAUTH_URL=https://<DNS_DO_SECP>
AUTH_TRUST_HOST=true

SECP_ADMIN_MATRICULA=secp
SECP_ADMIN_SENHA=<SENHA_INICIAL_TEMPORARIA>
SECP_ADMIN_NOME=Administrador SECP
SECP_ADMIN_EMAIL=<EMAIL_SUPORTE>

AD_AUTH_URL=<URL_DA_API_DE_LOGIN_AD>
NODE_ENV=production
APP_TIMEZONE=America/Manaus
TZ=America/Manaus

SARH MOCK=false
SARH_BASE_URL=<URL_BASE_SARH>
SARH_API_BASE_URL=<URL_API_SARH>
SARH_API_TOKEN=<TOKEN_SARH_SE_APLICAVEL>
SARH_TIMEOUT_MS=30000
SARH_ORACLE_HOME=/opt/oracle/instantclient

SECP_EQUIPAMENTO_WEBHOOK_TOKEN=<TOKEN_FORTE_WEBHOOK_EQUIPAMENTOS>
REDIS_HOST=redis
```

```

REDIS_PORT=6379
REDIS_URL=redis://redis:6379
AFD_UPLOAD_DIR=import/_upload/afd

BIOMETRIA_FACIAL_ENCRYPTION_KEY=<CHAVE_BASE64_32_BYTES>
BIOMETRIA_FACIAL_TEMPLATE_PEPPER=<PEPPER_FORTE>
BIOMETRIA_FACIAL_ALLOW_RAW_IMAGE_STORAGE=false

SECP_AUTO_WORKERS=false
AFD_AUTO_WORKER=false
REPROCESSAMENTO_GLOBAL_AUTO_WORKER=false
HENRY_COLETA_AUTO_WORKER=false
HENRY_ONLINE_AUTO_WORKER=false
CALENDARIO_INSTITUCIONAL_AUTO_WORKER=false
SARH_LOGIN_SYNC_AUTO_WORKER=false
SARH_SYNC_AUTO_WORKER=false

LOG_LEVEL=info
DOCKER_SOCKET_GID=<GID_DO_DOCKER_SOCKET>

```

Gerar segredos

```

openssl rand -hex 32
openssl rand -base64 32
stat -c '%g' /var/run/docker.sock

```

Use o GID retornado pelo ultimo comando em **DOCKER_SOCKET_GID**. Isso permite que a tela administrativa leia o estado dos containers quando necessario.

6. PgBouncer e Oracle Instant Client

PgBouncer

Antes de subir a stack, crie **docker/pgbouncer/userlist.txt** fora do Git.

```

cd /opt/secp/secp-app
printf '%s%s' '<SENHA_FORTE_DO_POSTGRES>' 'secp' | md5sum
sudo install -m 600 /dev/null docker/pgbouncer/userlist.txt
printf '"secp" "md5<HASH_GERADO>"\n' > docker/pgbouncer/userlist.txt

```

O valor final precisa ser a palavra **md5** concatenada com o hash MD5 de senha + usuario.

Oracle Instant Client para SARH

- Se o build tiver internet, o Dockerfile baixa automaticamente o pacote Linux x64 da Oracle.
- Para unidade sem internet no build, coloque o arquivo **docker/oracle/instantclient-basiclite-linuxx64.zip** antes de construir as imagens.
- Nao use client Windows dentro do container Linux.

7. Subida Inicial em Producao

Validar compose

```

cd /opt/secp/secp-app
export DOCKER_SOCKET_GID=$(stat -c '%g' /var/run/docker.sock)
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production config
>/tmp/secp-prod-compose.yaml

```

Construir imagens

```
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production build
```

Subir infraestrutura basica

```

docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production up -d postgres
redis pgbouncer
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production ps

```

Executar migrations e seed inicial

```

docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production --profile tools
run --rm migrate
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production --profile tools
run --rm seed

```

Execute seed inicial apenas na primeira implantacao ou quando houver orientacao explicita. Em atualizacoes comuns, migrations bastam.

Subir aplicacao e workers

```
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production up -d web
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production --profile
workers up -d
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production ps
```

8. Validacao Tecnica Pos-Subida

Saude da aplicacao

```
curl -fsS http://127.0.0.1:3000/api/health
curl -fsS http://127.0.0.1:3000/api/ready
docker inspect secp-web --format '{{.State.Health.Status}}'
```

Banco, Redis e PgBouncer

```
docker exec secp-postgres pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB" -h localhost
docker exec secp-redis redis-cli ping
docker exec secp-pgbouncer pg_isready -h 127.0.0.1 -p 6432 -U "$POSTGRES_USER" -d
"$POSTGRES_DB"
```

Logs essenciais

```
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production logs --tail=120
web
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production --profile
workers logs --tail=120 worker-sarh
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production --profile
workers logs --tail=120 worker-coleta-relogio
```

Checklist funcional

- Acessar a URL oficial do SECP e autenticar com usuario administrador inicial.
- Alterar a senha inicial temporaria ou desabilitar acesso local conforme politica da unidade.
- Validar cadastro de orgao/seccional, unidades, perfis e permissao administrativa.
- Executar sincronizacao SARH em lote pequeno e conferir servidores, afastamentos e ferias.
- Cadastrar um equipamento biometrico de teste e confirmar coleta de marcacoes brutas.
- Abrir /espelho-ponto para servidor conhecido e conferir competencia atual.

9. Publicacao por DNS ou Proxy Reverso

O compose publica o SECP na porta **3000** do host. Em producao, recomenda-se publicar via DNS interno e proxy reverso com TLS institucional.

Exemplo de proxy generico

```
# Exemplo conceitual. Adapte para IIS/ARR, Nginx, Caddy ou balanceador institucional.
https://secp.<unidade>.jus.br -> http://IP_DO_SERVIDOR:3000
```

```
Cabecalhos importantes:
X-Forwarded-Proto: https
X-Forwarded-Host: secp.<unidade>.jus.br
X-Forwarded-For: <IP_CLIENTE>
```

Depois de publicar, ajuste **AUTH_URL** e **NEXTAUTH_URL** para a URL oficial HTTPS e recrie o container web.

```
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production up -d --no-deps
--force-recreate web
```

10. Integracoes

SARH

- Confirmar conectividade do servidor SECP ate o SARH/API ou Oracle, conforme ambiente da unidade.
- Manter **SARH MOCK=false** em producao.

- Validar rotinas separadas quando disponiveis: servidores, afastamentos, ferias e demais endpoints.
- Acompanhar logs dos containers **secp-worker-sarh** e **secp-worker-sarh-login**.

Equipamentos biometricos

- Confirmar IP, porta, fabricante, modelo, orgao e unidade antes da primeira coleta.
- Para relógios Control iD FACE ID, usar protocolo Access API quando o endpoint **load_objects.fcgi** responder.
- Para Control iD idClass Bio, usar protocolo REP/AFD em HTTPS 443 quando **get_afd.fcgi** responder.
- Se o relógio pertencer a um orgao, o processamento deve normalizar matriculas numericas com o prefixo do orgao.
- Acompanhar **marcacoes_brutas**, eventos do equipamento e logs do **worker-coleta-relogio**.

AD/autenticacao

- Validar a URL **AD_AUTH_URL** a partir do servidor.
- Definir plano de contingencia para login local administrativo.
- Evitar expor credenciais LDAP ou tokens em chamados, prints ou scripts compartilhados.

11. Backup e Restauracao

Backup manual do PostgreSQL

```
cd /opt/secp/secp-app
mkdir -p backups
docker exec secp-postgres pg_dump -U secp -d secp_prod -Fc > backups/secp_prod_$(date +%Y%m%d_%H%M%S).dump
```

Backup de arquivos persistentes

```
docker run --rm -v secp-prod_secp_uploads_data:/data -v "$PWD/backups:/backups" alpine tar
  czf /backups/uploads_$(date +%Y%m%d_%H%M%S).tar.gz -C /data .

docker run --rm -v secp-prod_secp_relatorios_data:/data -v "$PWD/backups:/backups" alpine
  tar czf /backups/relatorios_$(date +%Y%m%d_%H%M%S).tar.gz -C /data .
```

Restauracao em ambiente controlado

```
# Exemplo: restaurar dump em banco vazio. Nao executar em producao sem janela e aprovacao.
docker exec -i secp-postgres pg_restore -U secp -d secp_prod --clean --if-exists <
  backups/ARQUIVO.dump
```

Regra operacional: antes de migrations, atualizacoes relevantes ou manutencoes no banco, gerar backup e validar que o arquivo foi criado com tamanho plausivel.

12. Atualizacao de Versao

- 1 Registrar versao atual: **git rev-parse HEAD** e **docker images | grep secp**.
- 2 Gerar backup do PostgreSQL e dos volumes de uploads/relatorios.
- 3 Atualizar codigo para tag ou commit aprovado.
- 4 Atualizar **APP_VERSION** no **.env.production**.
- 5 Rodar **docker compose config** para validar variaveis.
- 6 Construir imagens e executar migrations.
- 7 Recriar web e workers.
- 8 Validar health, ready, login, SARH, espelho de ponto e coleta de relógio.

```
cd /opt/secp/secp-app
git fetch --all --tags
git checkout <TAG_OU_COMMIT_APROVADO>
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production build
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production --profile tools
  run --rm migrate
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production up -d --no-deps
  --force-recreate web
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production --profile
  workers up -d --no-deps --force-recreate
```

13. Rollback

Rollback deve ter escopo claro. Se houve migration destrutiva, o rollback de imagem pode nao bastar; talvez seja necessario restaurar backup.

Rollback de imagem/codigo sem restaurar banco

```
cd /opt/secp/secp-app
git checkout <TAG_OU_COMMIT_ANTERIOR>
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production build web
worker-afd worker-sarh worker-coleta-relogio
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production up -d --no-deps
--force-recreate web
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production --profile
workers up -d --no-deps --force-recreate
```

Rollback com restauracao de banco

- Parar web e workers para evitar escrita durante restauracao.
- Restaurar dump aprovado em janela de manutencao.
- Subir web e workers da versao compativel com o dump.
- Validar login, health, ready e dados essenciais.

14. Observabilidade Opcional

O projeto possui stack de observabilidade com Prometheus, Grafana, Alertmanager, Tempo, Loki, Promtail e exporters.

Preflight

```
cd /opt/secp/secp-app
bash scripts/observability/preflight.sh
```

Preparar secrets e subir

```
cd /opt/secp/secp-app/observability
cp .env.example .env
mkdir -p secrets
openssl rand -hex 32 > secrets/secp_metrics_token
openssl rand -base64 32 > secrets/grafana_admin_password
printf 'https://SEU-WEBHOOK-ALERTMANAGER.example/secp\n' > secrets/alertmanager_webhook_url
bash scripts/validate.sh
bash scripts/start.sh
bash scripts/status.sh
```

O endpoint `/api/metrics` exige token Bearer. Nao publique Prometheus ou Grafana sem autenticao e controle de rede.

15. Operacao Diaria e Diagnostico Rapido

Comandos de rotina

```
docker compose -p secp-prod -f compose.prod.yaml --env-file .env.production ps
docker stats --no-stream
df -h
docker system df
curl -fsS http://127.0.0.1:3000/api/health
curl -fsS http://127.0.0.1:3000/api/ready
```

Quando a aplicacao nao responde

- 1 Verificar se **secp-web** esta Up/healthy.
- 2 Consultar logs recentes do **web**.
- 3 Validar PostgreSQL, PgBouncer e Redis.
- 4 Verificar uso de disco e memoria.
- 5 Testar acesso local **curl http://127.0.0.1:3000/api/health** antes de culpar proxy ou DNS.

Quando filas ou workers atrasam

- Verificar logs do worker especifico.

- Conferir Redis com **redis-cli ping**.
- Avaliar se relógios biométricos estão online e se o cursor NSR está muito atrasado.
- Evitar aumentar concorrência sem medir CPU, memória, banco e rede.

Quando o banco apresenta erro

- Testar **pg_isready** no PostgreSQL e no PgBouncer.
- Não rodar **vacuum full**, migrations manuais ou reset sem backup e janela.
- Verificar conexões, tamanho do banco e logs do container.

16. Checklist de Entrega da Implantação

Item	Validado	Observação
Servidor Ubuntu atualizado e com Docker Compose v2	()	
Repositório em tag/commit aprovado	()	
.env.production criado sem valores padrão inseguros	()	
PgBouncer userlist.txt criado	()	
Migrations executadas com sucesso	()	
Web e workers rodando	()	
/api/health e /api/ready respondendo	()	
Login administrativo validado	()	
SARH validado em lote pequeno	()	
Equipamento biométrico de teste validado	()	
Backup inicial gerado e armazenado	()	
DNS/proxy/TLS validado	()	
Responsáveis de operação definidos	()	

Unidade	
Responsável técnico	
Data da implantação	
Commit/tag implantado	
URL oficial	